Dealing with Data

CSCI-SHU 11
Fall 2025

Lecture's Outline

- Data types
  - string
  - numerical values
    - integer, floating point, complex
  - Data type conversion
- Strings
  - indexing
  - slicing

Data Types

Values

- Values are data
  - for now, a value is a number or a string
    - we'll see more types later
  - it can be stored into a variable
  - it can be part of an expression
    - a combination of values, operators, functions,… that can be evaluated
  - but it can't be evaluated on its own
    - 2 + 3 is not a value, because it can be evaluated further
- Examples of values are:
  - -123456 is an integer literal
  - 'a string is a value' is a string literal

Data Types

- Classification of values
  - Values can be classifies by Data Type
- Today, we will go over the following 4 types:
  - str: string
  - int: integer
  - float: floating point
  - complex: complex number
- The last 3 types are numeric types

Strings

- A string is a sequence of characters:
  - any characters
  - including spaces and punctuation
  - must start and end with quotes! (string delimiters)
- String delimiters
  - single quotes: 'I am a string'
  - double quotes "I'm a string"
  - triple single quotes: '''I am a string'''
  - triple double quotes: """I'm a string"""

Strings

Examples

```
'Single quoted string'
```

```
"Double quoted string"
```

```
'''First line
Second line'''
```

```
"""First Line
Second Line
Third Line"""
```

Strings

Unbalanced Quotes

- When you define a string, you need to be careful with the starting and ending quotes.
- For example, an extra quote in the string will lead to a Syntax Error since it is ambiguous which quote is the ending one.
- Example:

```
>>>'I'm a string'
>>>SyntaxError: unterminated string literal
(detected at line 1)
```

Strings

Workarounds

- Workaround 1: Mixing quotes
  - If your string contains a particular quote, you have to choose another quote for string delimiters
  - Example:

```
>>>"I'm a string"
>>>
```

- Workaround 2: Escape sequence

Escape Sequence

  - Escape sequence can be used to obtain quote character in a string
  - Escape sequence starts with a backslash symbol \
  - Example:

```
>>>'I\'m a string'
```

Strings

Escape Sequence	
`\\`	Backslash (\)
`\'`	Single quote (')
`\"`	Double quote (")
`\n`	ASCII Linefeed (LF)
`\r`	ASCII Carriage Return (CR)
`\t`	ASCII Horizontal Tab (TAB)
`\v`	ASCII Vertical Tab (VT)

Numeric Types

- The following types are all closely related and most of the same operations can be applied:

`int`	Integer
`float`	Floating point
`complex`	Complex number

Numeric Types

Integer

- Integer: whole number, can be negative
- No size limit (as much as your computer can handle)!

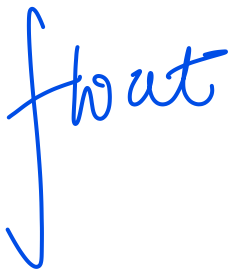
- Examples
  - `24`
  - `-61`
  - `16 * 88`
  - `1337 ** 20`
  - `2456 ** 10000`

Numeric Types

Floating point

- A floating point number represents a decimal fraction
- It differs from integer with the presence of a decimal point

- Examples
  - `-5.5555` is a float
  - `14.0` is a float
  - `14` is a integer



Numeric Types

Scientific notation

- You can input float numbers using scientific notation using *e* for decimal exponentiation

- Examples
  - `2e+3` is $2*10^{3}$
  - `4e-5` is $4*10^{-5}$
  - `123.456e100` is $123.456*10^{100}$

科学计数法

Numeric Types

Float Accuracy

- A float is also coded with binary digits
  - It uses some negative powers of 2.
- Representation of 0.2 as a float requires a infinite number of bits:

$$0.2 = 0.125 + 0.0625 + 0.0078125 + 0.00390625 + \ldots$$

$$= 2^{(-3)} + 2^{(-4)} + 2^{(-7)} + 2^{(-8)} + \ldots$$

  - Floats are thus truncated

Numeric Types

Complex numbers

- Python also supports complex numbers
  - numbers with a real and an imaginary part
- The imaginary part is indicated with character *j* (square root of -1)

虚数

- Example:
  - `1j`
  - `2 - 3j`
  - `2.4 + 3.6j`
  - `1j*1j`

Numeric Types

Mixing Numeric Types

- Python is able to mix numeric types during arithmetic operations.
- The operand with the "narrower" type will be widened to that of the other:

int ⊂ float ⊂ complex

整数 浮点数 复数

- Examples:

`2`	Integer	`2 ** 3`	Integer
`3.0`	Float	`2 ** 3.0`	Float
`1.1 + 5j`	Complex	`2 ** -3`	Float
`2 + 3.0`	Float	`2 ** 3j`	Complex
`2 * 3`	Integer	`1j * 1j`	Complex
`2 * 3.0`	Float	`2 / 3`	Float
`2 * -3`	Integer	`6j / 3j`	Complex

Data Types

- We can use the built-in function *type*
- It displays in the shell what type the value is

- Examples:

数据类型

`type(5)`	`<class 'int'>`
`type(2.0)`	`<class 'float'>`
`type('hello')`	`<class 'str'>`

Data Type Conversion

- Python provides several builtin functions that can convert Data types into other types
- The functions are:
  - int()
  - float()
  - complex()
  - str()
    - *str()* can convert an *int*, a *float*, or a *complex* into a *str*

Convert 内置函数转换数据类型.

Data Type Conversion

```
                    int()
>>>int("123")              >>>int(456.78)

                    float()
>>>float("123.456")        >>>float(5)

                    complex()
>>>complex("1 + 2j")       >>>complex(5)

>>>complex(5.1, 6.4)

                    str()
>>>str(123.456)             >>>str(5)


>>>str(1+2j)
```

Data Type Conversion

int()

```
>>>int("123")
123
```

```
>>>int(456.78)
456
```

float()

```
>>>float("123.456")
123.456
```

```
>>>float(5)
5.0
```

complex()

```
>>>complex("1 + 2j")
(1 + 2j)
```

```
>>> complex(5)
(5 + 0j)
```

```
>>>complex(5.1, 6.4)
(5.1 + 6.4j)
```

str()

```
>>>str(123.456)
'123.456'
```

```
>>> str(5)
'5'
```

```
>>>str(1+2j)
'(1+2j)'
```

X "1+2j"

Comments

注释.

- A comment is a text in a program that is meant for the human reader
  - it is ignored by the interpreter.
- A comment starts with a hashtag character #
- The interpreter will ignore everything from that character to the end of the line
- Example:

```
print("This is an example")              #first printed line
print("Comments won't be displayed")     #second printed line
```

```
This is an example
Comments won't be displayed
```

Operations

Addition symbol +

- Addition symbol +
  - can add operands with **same** numeric types
  - can add operands with **different** numeric types
  - can add strings: concatenation
  - cannot add a string with a numeric value
    - doing so leads to TypeError
- Examples:

```
>>>2 + 3
5

>>>2.3 + 1.7
4.0
```

```
>>>1 + 6.4
7.4

>>>'Concatenating' + ' ' + 'strings'
'Concatenating strings'
```

Operations

Multiplication symbol *

- Addition symbol *
  - can multiply operands with same numeric types
  - can multiply operands with different numeric types
  - cannot multiply strings
  - can multiply a string with an integer: Repetition
    - other numeric types lead to TypeError
- Examples:

```
>>>2 * 3
6

>>>2.3 * 1.7
3.91
```

```
>>>10 * 6.4
64.0

>>>'Hey' * 3
'HeyHeyHey'
```

Operations

Integer division symbol //

- Integer division symbol //
  - Always returns the integer part (nearest integer on the left) of the division
    - can be output on a *float depending on the operand types*
  - can be used with *int* or *float* operands (mixed or not)
  - cannot be used with *str* or *complex* operands
- Examples:

```
>>>5 // 2
2

>>>2.3 // 1.7
1.0
```

```
>>>-5 //2
-3

>>>10 // 2.5
4.0
```

// 整除.

向下取整.

可以是浮点结果.

Operations

Modulo symbol %

- Modulo symbol %
  - Returns the remainder of the division
  - can be used with *int* or *float* operands (mixed or not)
  - cannot be used with *str* or *complex* operands
- Examples:

```
>>>5 % 2
1

>>>8.32 % 1.3
0.52
```

```
>>>-5 % 2
1

>>>10 % 2.5
0.0
```

Operations

Exponentiation symbol **

- Exponentiation symbol **
  - can be used with any numeric type (mixed or not)
  - cannot be used with *str* operands
- Examples:

```
>>>5 ** 4
625

>>>-5.5 ** 2
-30.25
```

```
>>>2.5 ** 3.0
15.25

>>>8.32 ** 4.5
13821.4933238
```

Operations

Operators precedence

- List of operators from highest to lowest precedence:
  1. \- ** exponentiation
  2. \- + or - sign
  3. \- * / // % multiplication, division, integer division and modulo
  4. \- + - addition and substraction
- Parentheses
  - In order to change the order or operations, you should use parentheses to group expressions
- Examples:

```
>>> 6 + 4 * 5
26
```

```
>>> (6+4) * 5
50
```

Operations Summary

Operations on numeric values		
Symbol	Operation	
+	addition	
–	substraction	
*	multiplication	
/	division	always returns a float (or complex)
//	integer division	cannot use complex
%	modulo	cannot use complex
**	exponentiation	

Operations on strings		
Symbol	Operation	
+	concatenation	both operands must be a string
*	repetition	one operand must be a str, the other an int

Strings

Indexing & Slicing

切片操作，取数

Strings 字符串

- String delimiters
  - 'single quotes'
  - "double quotes"
  - """Triple double quotes"""
- Strings vs Variables

```
>>>foo = "bar"
>>>print(foo)
"bar"
>>>print("foo")
"foo"
```

Strings

String Operations

- Some operations that we can perform on strings:
  - addition/concatenation
  - multiplication/repetition
  - string formatting (old style)

+	addition/concatenation
*	multiplication/repetition
%	string formatting (old style)

Strings

String vs Characters

- Strings revisited
  - we have been looking at strings as single things (single objects)
  - but, they are made of a bunch of smaller pieces
  - so, what are strings made of?
    - zero or more characters!
- Characters in a string
  - strings are specifically a sequence of characters
  - that is, they are ordered

Strings

Compound Data Type

- A string is a compound data type
  - a compound data type is for values composed of zero or more other values
- Examples of compound data types include:
  - ranges,
  - lists, 列表
  - tuples 元组
  - sets, 集合
  - dictionaries 字典
  - …

Strings

Unicode Code/Text Conversion

- The ord() function returns
  - the Unicode code for one-character string
- The chr() function returns
  - the Unicode text of one character with the provided unicode code

```
>>>ord('a')
97

>>>ord('A')
65

>>>ord('\n')
10
```

'a'-97

'A'-65

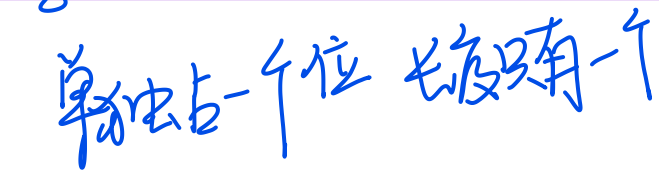
```
>>>chr(97)
'a'

>>>chr(65)
'A'

>>>chr(9)
'\t'
```

Strings

len() function

- The len() function returns the length of a string
  - that is the number of characters it contains

```
>>>len("0123456789")
10

>>>len("")
0

>>>len("\n")
1
```



Strings

Indexing String

- Each character of a string has an index value
  - an index is a numeric value that specifies the position of an item in an ordered collection
  - each character can be referred to by its index (its place in the string)
  - indexes start at 0 (not at 1)

Index	0	1	2	3	4	5	6	7	8	9	10	11
String	H	e	l	l	o		W	o	r	l	d	!

```
>>>len("Hello World!")
12
>>>len("")
0

>>>len("\n")
1
```

Strings

Indexing Strings

- You can reference a specific character in a string (indexing) by:
  - using the index an integer
    - or a variable the index value has been assigned to
  - enclosed by square brackets: [ ]
  - immediately following a string
    - or a variable the string has been assigned to
  - returns a new string consisting of only that one character

```
>>> len("0123456789")
10

>>> "0123456789"[0]
'0'

>>> one_digit_numbers = "0123456789"

>>> len(one_digit_numbers)
10

>>> one_digit_numbers[0]
'0'
```

Strings

Indexing Strings

- You can reference a specific character in a string (indexing) by:
  - using the index an integer
    - or a variable the index value has been assigned to
  - enclosed by square brackets: [ ]
  - immediately following a string
    - or a variable the string has been assigned to
  - returns a new string consisting of only that one character

```
>>> one_digit_numbers = "0123456789"

>>> one_digit_numbers[0]
'0'

>>> len(one_digit_numbers)
10

>>> one_digit_numbers[10]
Traceback (most recent call last):
  File "<pyshell#167>", line 1, in <module>
    one_digit_numbers[len(one_digit_numbers)]
IndexError: string index out of range
```

=> 超出范围了. index out of range

Strings

Indexing Strings

- You can reference a specific character in a string (indexing) by:
  - using the index an integer
    - or a variable the index value has been assigned to
  - enclosed by square brackets: [ ]
  - immediately following a string
    - or a variable the string has been assigned to
  - returns a new string consisting of only that one character

```
>>> one_digit_numbers = "0123456789"

>>> one_digit_numbers[0]
'0'

>>> len(one_digit_numbers)
10

>>> one_digit_numbers[len(one_digit_numbers)-1]
'9'
```

Strings

Indexing String

```
>>> alphabet = "abcdefghijklmnopqrstuvwxyz"

>>> len(alphabet)
26

>>> alphabet[0]
'a'

>>> alphabet[len(alphabet)-1]
'z'

>>> "alphabet"[len("alphabet"-1)]
't'

>>> alphabet[ord('a')-97]
'a'
```

Strings

Negative indexes

- You can index strings from the end with negative indexes:
  - -1 is the index of last character
  - -2 second to last
  - …

Positive index	0	1	2	3	4	5	6	7	8	9	10	11
Negative index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
String	H	e	l	l	o		W	o	r	l	d	!

```
>>> hello_world = "Hello World!"

>>> hello_world[-1]
'!'

>>> hello_world[-7]
' '
```

Strings Are Immutable

#字符串是不可改变的 immutable 不可改变的

- Strings are not mutable
  - you can read values by indexing into a string
  - you cannot change values
  - you cannot set an indexed character equal to another character

```
>>> hello_world_1 = "Hello World"

>>> hello_world_1[6] = 'w'
Traceback (most recent call last):
  File "<pyshell#178>", line 1, in <module>
    hello_world[-1] = '?'
TypeError: 'str' object does not support item assignment
```

#不能赋值

Strings Are Immutable

- Strings are not mutable
  - you can read values by indexing into a string
  - you cannot change values
  - you cannot set an indexed character equal to another character

```
>>> hello_world_1 = "Hello World"

>>> hello_world_2 = hello_world_1 + '?'
>>> print(hello_world_2)
Hello World?

>>> hello_world_1 = hello_world_2
>>> print(hello_world_1)
Hello World?
```

Slicing

- Slicing is the act of extracting a substring from a given string
- The slicing string syntax is the following:

string_object[start:end:step]

- where:
  - *start* is the start position
  - *end* is the stop position (character at the stop position **not included**)
  - *step* is the increment (value 1 by default)

```
>>> slice_me = "ABCDEF123456"
>>> len(slice_me)
12

>>> slice_me[0:6]
'ABCDEF'

>>> slice_me[6:len(slice_me)]
'123456'
```

Slicing

Negative Indices

- You can also specify negative indices while slicing a string

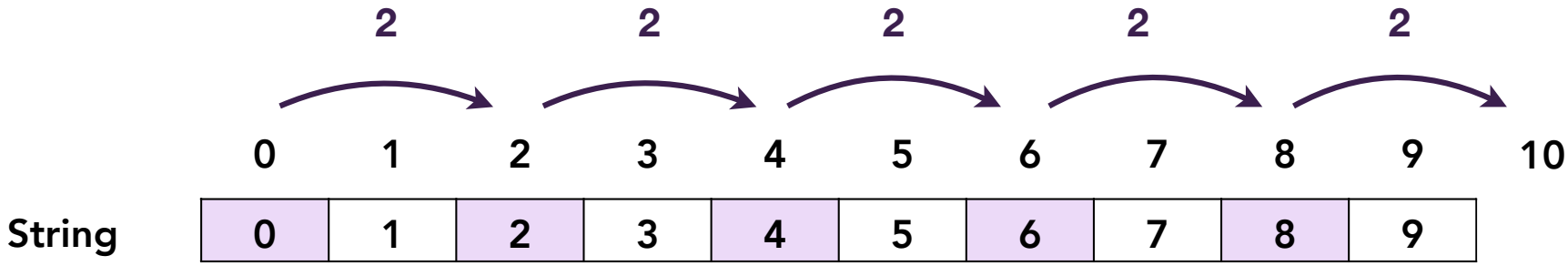
Positive index	0	1	2	3	4	5	6	7	8	9	10	11
Negative index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
String	A	B	C	D	E	F	1	2	3	4	5	6

```
>>> slice_me = "ABCDEF123456"

>>> slice_me[-12:-6]
'ABCDEF'

>>> slice_me[0:-6]
'ABCDEF'

>>> slice_me[-9:9]
'DEF123'
```

Slicing

Slicing Steps

- You can specify the step of the slicing using the step parameter
  - The step parameter is optional and by default 1



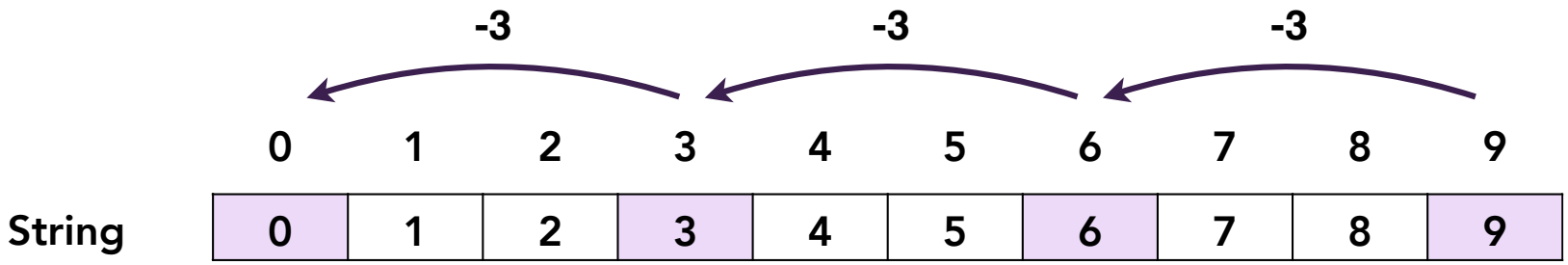

```
>>> slice_me = "0123456789"

>>> slice_me[0:10:2]
'02468'

>>> slice_me[1:10:2]
'13579'

>>> slice_me[1:9:2]
'1357'
```

```
>>> slice_me[0:9:3]
'036'

>>> slice_me[0:10:3]
'0369'
```

Slicing

Slicing Steps

- You can specify the step of the slicing using the step parameter
  - The step parameter is optional and by default 1
  - The step parameter can be negative




```
>>> slice_me[9:0:-2]
'97531'

>>> slice_me[10:1:-2]
'9753'

>>> slice_me[9:0:-3]
'963'

>>> slice_me[9:0:-1]
'987654321'
```

```
>>> slice_me[-1:-10:-2]
'97531'

>>>slice_me[10:1:-2]
'9753'

>>>slice_me[0:-1:2]
'02468'

>>>slice_me[-1:0:-2]
'97531'
```

Slicing

Slice at Beginning & End

- start and end indices can be omitted
  - Omitting the start index starts the slice from the index
    - *string_object[:stop]* is equivalent to *string_object[0:stop]*
  - Omitting the stop index extends the slice to the end of the string.
    - *string_object[start:]* is equivalent to *string_object[start:len(string_object)]*

Positive index	0	1	2	3	4	5	6	7	8	9	10	11
Negative index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
String	A	B	C	D	E	F	1	2	3	4	5	6

```
>>> slice_me = "ABCDEF123456"

>>> slice_me[:6]
'ABCDEF'

>>> slice_me[6:]
'123456'
```

```
>>>slice_me[:6:2]
'ACE'

>>> slice_me[6::2]
'135'
```

Slicing

Reversing a String

- Omitting both *start* and *end* indices and specifying the *step* as -1

Positive index	0	1	2	3	4	5	6	7	8	9	10	11
Negative index	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
String	A	B	C	D	E	F	1	2	3	4	5	6

```
>>> slice_me = "ABCDEF123456"

>>> slice_me[::-1]
'654321FEDCBA'

>>> slice_me[5::-1]
'FEDCBA'

>>> slice_me[5:0:-1]
'FEDCB'
```

```
>>> slice_me[:5:-1]
'654321'

>>> slice_me[-1:-7:-1]
'654321'
```

Takeways

- Basic data types in Python
  - Integers
  - Floating-Point Numbers
  - Complex Numbers
  - Strings
  - Boolean (introduced later in this course)
- Strings
  - Strings are indexed using positive numbers from 0 to the string length - 1
  - Substrings can be extracted from a larger string using slicing

Readings & Assignments

- Quiz 01 on Tue 09/08 Prepare!
- Preparation for Lecture 03
  - Read chapter 3 in the textbook
  - Watch self-paced modules 3:
    - https://stream.nyu.edu/playlist/dedicated/306991832/1_gm6t0b7p/